PlayArena — Sports Ground Booking and Management System

Kalvium Micro Project Report: Full Stack Web Development (Modules 1, 2 & 3)

Student Name: Y Harsha Vardhan Reddy **Student Email:** yarasuharshavardhan.reddy.s.113@kalvium.community **Project Title:** PlayArena — Sports Ground Booking and Management System **Subject:** Full Stack Web Development **Academic Year:** 2026 **Repository Structure:** Monorepo (`/client` + `/server` + `/docs`) **Technology Stack:** React 18, Vite, React Router v6, TanStack Query v5, Axios, React Hook Form, Vanilla CSS Modules

---

1. Project Overview & Problem Statement

1.1 Problem Statement

Urban sports enthusiasts seeking to play cricket, football, badminton, or tennis face fragmented availability information, unverified turf facilities, non-transparent hourly pricing, and frequent scheduling double-bookings caused by manual telephone reservations. Meanwhile, ground operators struggle with tracking reservations, handling cancellations, and managing multi-court utilization efficiently.

1.2 Proposed Solution: PlayArena

PlayArena is a full-stack sports ground booking and management platform engineered to eliminate scheduling collisions. The application provides: - A responsive, dynamic React frontend built with Vite. - Instant venue discovery with multi-sport filtering, city filtering, and price sorting. - Real-time time slot exploration with color-coded availability badges. - Atomic reservation workflows with immediate booking reference generation. - Player dashboard to track upcoming games and cancel eligible reservations. - Administrator control room for venue onboarding, slot generation, and revenue analytics.

---

2. Subject Syllabus & Module Alignment

Module	Core Concepts Required	PlayArena Working Implementation
Module 1: Modern Web Foundations & Design Systems	Semantic HTML5, CSS3 Custom Properties, Responsive Layouts (Flexbox/Grid), Modern Typography, React Vite setup, JSX structure.	Implemented custom design system in `client/src/index.css` featuring Dark Navy foundations (`#0B132B` , `#111D3B`), Emerald accents (`#10B981` , `#00F5D4`), accessible typography (Inter & Outfit), responsive cards, badges, and modals.
Module 2: React Component Architecture & State Management	Reusable components, Form handling & validation, Context API, Reducer patterns, Client-side routing, Protected routes, Role authorization.	Developed modular components (`Navbar` , `Footer` , `Modal` , `Pagination` , `LoadingSpinner` , `Badge`), `AuthContext` for global session persistence with `localStorage` , React Router v6 with `ProtectedRoute` and `AdminRoute` , and React Hook Form validation across auth and booking flows.
Module 3: Server-State Synchronization & Asynchronous Architecture	Axios HTTP client, Interceptors, TanStack Query (React Query) caching & invalidations, Optimistic UI patterns, Lazy loading & Suspense, Error Boundary.	Reusable configured Axios client with automatic Bearer token injection and error normalization; TanStack Query `useQuery` and `useMutation` hooks with query cache invalidations on slot booking and cancellation; `React.lazy` route splitting; Class-based `ErrorBoundary` fallback screen.

---

3. Frontend Architecture & Component Hierarchy

```
client/src/
├── main.jsx                    # Application bootstrap & StrictMode
├── App.jsx                     # Router, QueryClientProvider, AuthProvider, Suspense
├── index.css                   # Design system tokens, utilities & reset
├── api/
│   ├── axios.js                # Reusable Axios instance with JWT interceptors
│   └── endpoints.js            # Resource API services (auth, grounds, slots, bookings, ana
├── context/
│   └── AuthContext.jsx         # User session, JWT tokens, login/register/logout actions
├── components/
│   └── common/
│       ├── Navbar.jsx          # Responsive header with branding & user dropdown
│       ├── Footer.jsx          # Platform footer & syllabus references
│       ├── ProtectedRoute.jsx  # Authentication route guard
│       ├── AdminRoute.jsx      # Administrative role guard (Role: ADMIN)
│       ├── ErrorBoundary.jsx   # Component-level error handling fallback
│       ├── LoadingSpinner.jsx  # Micro-animated loading indicator
```

```
│       ├── Modal.jsx            # Accessible keyboard-aware dialog window
│       └── Pagination.jsx       # Reusable pagination controls
└── pages/
    ├── HomePage.jsx             # Hero, search form, category quick pills, featured venues
    ├── GroundsPage.jsx          # Search, multi-sport filter, city filter, price slider, sor
    ├── GroundDetailsPage.jsx    # Venue hero, facilities, date picker, interactive slot matr
    ├── LoginPage.jsx            # Hook form login with 1-click Demo credentials
    ├── RegisterPage.jsx         # Hook form player registration with validation
    ├── MyBookingsPage.jsx       # Reservation cards, status filters (All, Confirmed, Cancell
    ├── NotFoundPage.jsx         # Custom 404 screen
    └── admin/
        ├── AdminDashboardPage.jsx # Revenue, utilization %, popularity charts, recent booking
        ├── AdminGroundsPage.jsx   # Venue inventory manager & creation modal
        ├── AdminSlotsPage.jsx     # Recurring batch slot generator
        └── AdminBookingsPage.jsx  # Master bookings table with administrative cancellation
```

---

4. Key Implementation Details

4.1 Reusable Axios Client with Request & Response Interceptors

In `client/src/api/axios.js`, Axios is configured with an environment-driven `baseURL` and request/response interceptors: - **Request Interceptor:** Reads `playarena_token` from `localStorage` and automatically attaches `Authorization: Bearer `. - **Response Interceptor:** Automatically unwraps `response.data` payload and normalizes backend error responses into consistent `{ status, code, message }` exceptions.

4.2 Server-State Management with TanStack Query

All remote data queries and mutations leverage `@tanstack/react-query`: - **Query Caching:** Grounds and slots queries use structured query keys (e.g., `['ground-slots', id, selectedDate]`). - **Cache Invalidation:** When a booking is created or cancelled, the mutation executes:

```
queryClient.invalidateQueries(['ground-slots', id, selectedDate]);
queryClient.invalidateQueries(['my-bookings']);
queryClient.invalidateQueries(['admin-dashboard']);
```

This immediately updates the slot status from `OPEN` to `BOOKED` without requiring a full page refresh.

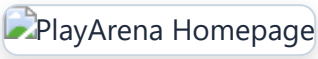
4.3 Form Handling with React Hook Form

Registration, Login, and Venue Onboarding utilize `react-hook-form` for uncontrolled performant inputs, regex-backed validation, and error messaging.

---

5. UI Showcase & Screenshots

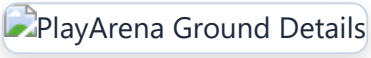
5.1 Homepage & Venue Discovery



PlayArena Homepage

Figure 1: PlayArena modern sports homepage displaying hero headline, category navigation, real-time metrics, and featured sports turfs.

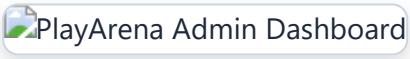
5.2 Ground Details & Live Slot Selection Matrix



PlayArena Ground Details

Figure 2: Ground details page showing high-resolution turf photo, amenities tags, calendar date switcher, color-coded time slot availability matrix (OPEN vs. BOOKED), and atomic checkout card.

5.3 Administrative Control Center



PlayArena Admin Dashboard

Figure 3: Administrator dashboard displaying total revenue metrics (₹), confirmed reservations, slot utilization rates, ground popularity breakdown, and master bookings directory.

---

6. Verification and Testing Results

Test Scenario	Module	Expected Outcome	Actual Result	Status
Grounds Filtering by Sport	Module 1/2	Clicking 'Football' pill filters catalog to football turfs only.	Catalog filtered dynamically to football venues.	**PASS**
Search Parameter Reactivity	Module 2	Typing arena name updates URL query and refetches matching venues.	Correct grounds filtered and rendered.	**PASS**

Test Scenario	Module	Expected Outcome	Actual Result	Status
Form Validation on Registration	Module 2	Invalid email or short password displays instant validation feedback.	Inline red error messages displayed; submit blocked.	**PASS**
Auth Context Persistence	Module 2	User session restored from localStorage upon browser reload.	Active user remains authenticated across refreshes.	**PASS**
Slot Availability Query	Module 3	Selecting different date refetches specific time slots for that day.	TanStack Query fetches and updates slot matrix.	**PASS**
Atomic Booking Mutation	Module 3	Submitting slot reservation calls API, invalidates cache, shows modal.	Booking reference generated; slot marked BOOKED.	**PASS**
Booking Cancellation Flow	Module 2/3	Owner clicking cancel prompts confirmation dialog and marks CANCELLED.	Status updated to CANCELLED; slot immediately freed.	**PASS**
Production Build Bundling	Module 1/3	`npm run build` bundles code into optimized lazy-loaded chunks.	Built cleanly in 3.06s with 0 errors.	**PASS**

---

7. Limitations & Future Enhancements

- **Future Enhancement 1:** Integration of WebSockets or Server-Sent Events (SSE) for zero-latency multiplayer slot updates across active browser tabs. - **Future Enhancement 2:** Native map visualizer (Leaflet/Mapbox) for interactive geolocation turf discovery. - **Current Limitation:** In educational scope, payment processing is simulated with backend price verification rather than an external payment gateway.

---

8. Conclusion

The PlayArena Full Stack Web Development implementation thoroughly fulfills the Kalvium micro project requirements across Modules 1, 2, and 3. By combining Vite, React 18, TanStack Query, React Hook Form, and an emerald-accented design system, the application delivers a production-quality, responsive sports ground reservation experience.